

DeskForge + BeatSync



Alan Bissio

E.E.S.T. N°1 Vicente López

7mo 2da

Proyecto de Implementación de Sitios Web Dinámicos

Jorge Guzman

22 de abril de 2026

Índice

1. Introducción.....	3
2. Objetivo de la actividad.....	3
3. Tecnologías utilizadas.....	5
3.1. Electron.....	5
3.2. Node.js.....	5
3.3. HTML, CSS y JavaScript.....	6
3.4. Vite.....	6
3.5. Howler.js.....	7
3.6. Web Audio API.....	7
3.7. Spotify Web API.....	8
3.8. pnpm.....	8
3.9. electron-builder.....	8
4. Arquitectura de la solución.....	10
5. Ventana principal y funcionalidades de escritorio.....	13
6. Bloc de Notas.....	14
7. Temporizador Pomodoro.....	15
8. Reproductor musical.....	16
9. Carga de música local.....	17
10. Visualizaciones de audio.....	18
11. Playlists.....	19
12. Integración con Spotify.....	20
13. Atajos globales.....	22
14. Ecualizador.....	23
15. Tema claro y oscuro.....	24
16. Empaquetado para Windows.....	25
17. Seguridad e IPC.....	26
18. Pruebas de funcionamiento.....	27
19. Dificultades encontradas.....	29
20. Resultado final.....	32
21. Conclusión.....	34

1. Introducción

En esta actividad se desarrolló **DeskForge + BeatSync**, una aplicación de escritorio orientada a combinar herramientas de productividad con un reproductor musical interactivo.

El proyecto fue desarrollado utilizando **Electron**, lo que permite construir aplicaciones de escritorio utilizando tecnologías web como HTML, CSS y JavaScript, pero incorporando también acceso a funciones propias del sistema operativo.

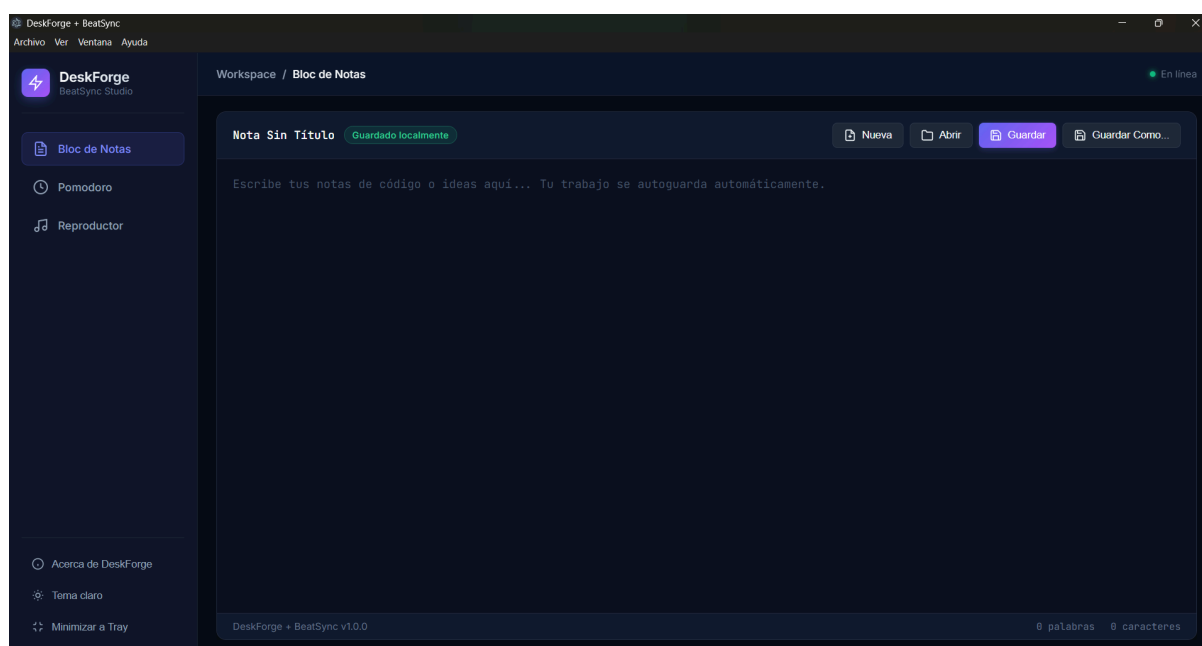
DeskForge incluye diferentes herramientas de productividad, entre ellas un bloc de notas y un temporizador Pomodoro. Por otra parte, BeatSync incorpora un reproductor de música local con soporte para distintos formatos de audio, playlists, visualizaciones en tiempo real, ecualizador e integración con Spotify.

Uno de los objetivos principales de la actividad fue trabajar con la arquitectura propia de Electron, separando el proceso principal de la aplicación, el preload y el renderer. De esta manera, las funciones que requieren acceso al sistema operativo pueden mantenerse separadas de la interfaz.

Además, se utilizaron APIs nativas de Electron para incorporar ventanas, diálogos, notificaciones, bandeja del sistema y atajos globales.

Para la reproducción de música se utilizó Howler.js, mientras que Web Audio API permitió analizar el audio y desarrollar diferentes visualizaciones y un ecualizador.

Finalmente, la aplicación fue empaquetada para Windows utilizando electron-builder, generando un instalador .exe.



2. Objetivo de la actividad

El objetivo principal de la actividad fue desarrollar una aplicación de escritorio utilizando Electron que combinara herramientas de productividad con un reproductor musical avanzado.

La implementación debía permitir:

- Crear una ventana principal mediante Electron.
- Incorporar un menú nativo.
- Utilizar la bandeja del sistema.
- Desarrollar un bloc de notas con apertura y guardado de archivos.
- Implementar un temporizador Pomodoro.
- Reproducir música mediante Howler.js.
- Cargar archivos de música desde carpetas locales.
- Reproducir archivos MP3, OGG y WAV.
- Crear visualizaciones de audio mediante Web Audio API.
- Incorporar playlists con sistema de arrastrar y soltar.
- Integrar una API externa como Spotify o Last.fm.
- Incorporar atajos globales.
- Implementar un ecualizador.
- Incorporar tema claro y oscuro.
- Generar un instalador de escritorio mediante electron-builder.

De esta manera, el proyecto permitió aplicar conceptos relacionados con desarrollo web, aplicaciones de escritorio, manejo de archivos, APIs externas, procesamiento de audio y comunicación entre procesos.

3. Tecnologías utilizadas

Para desarrollar DeskForge + BeatSync se utilizaron diferentes tecnologías y herramientas.

3.1. Electron

Electron fue utilizado como tecnología principal para convertir la aplicación web en una aplicación de escritorio.

La versión utilizada durante el desarrollo fue:

Electron 44.4.4

Electron permite utilizar Chromium para mostrar la interfaz y Node.js para trabajar con funciones propias del sistema operativo.

Dentro del proyecto se utilizó para:

- Crear la ventana principal.
- Crear el menú nativo.
- Crear la bandeja del sistema.
- Mostrar diálogos de archivos.
- Mostrar notificaciones.
- Registrar atajos globales.
- Abrir enlaces externos.
- Comunicar el proceso principal con la interfaz.
- Empaquetar la aplicación para Windows.

La configuración principal de Electron se encuentra en:

electron/main.js

3.2. Node.js

Node.js se utiliza junto con Electron para ejecutar el proceso principal de la aplicación.

La versión utilizada durante el desarrollo fue:

Node.js 22.23.2

Entre otras cosas, Node.js se utiliza para:

- Acceder al sistema de archivos.

- Leer carpetas de música.
- Guardar archivos del bloc de notas.
- Crear el servidor local utilizado durante la autenticación con Spotify.
- Administrar funciones propias del sistema operativo.

3.3. HTML, CSS y JavaScript

La interfaz de DeskForge fue desarrollada principalmente utilizando HTML, CSS y JavaScript.

HTML se utiliza para definir la estructura de las diferentes herramientas de la aplicación.

CSS se utiliza para mantener una interfaz común entre todas las secciones, incluyendo:

- Barra lateral.
- Botones.
- Tarjetas.
- Formularios.
- Reproductor.
- Visualizadores.
- Spotify.
- Ecualizador.
- Tema claro y oscuro.

JavaScript se utiliza para controlar las diferentes funcionalidades de la aplicación y comunicar la interfaz con los servicios internos.

Los archivos principales de la interfaz se encuentran dentro de:

src/

3.4. Vite

Vite fue utilizado como herramienta de desarrollo y compilación de la interfaz.

La versión utilizada fue:

Vite 8.3.0

Durante el desarrollo se utilizó para iniciar el servidor local mediante:

`pnpm dev`

También se utiliza durante el proceso de compilación final mediante:

pnpm build

Vite genera los archivos optimizados de producción dentro de la carpeta:

dist/

3.5. Howler.js

Howler.js fue utilizado para controlar la reproducción de audio dentro de BeatSync.

La versión utilizada fue:

Howler.js 2.2.4

Esta biblioteca permite controlar funciones como:

- Reproducir.
- Pausar.
- Cambiar de canción.
- Modificar volumen.
- Controlar la posición de reproducción.
- Detectar el final de una canción.

También permite trabajar con diferentes formatos de audio.

En el proyecto se utilizan principalmente:

- MP3.
- OGG.
- WAV.

3.6. Web Audio API

Web Audio API se utilizó para trabajar directamente con el audio reproducido.

Esta tecnología permitió implementar dos funciones principales:

- Visualizaciones de audio.
- Ecualizador.

Para las visualizaciones se utiliza un `AnalyserNode`, que permite obtener información de las frecuencias y de la forma de onda del audio en tiempo real.

Para el ecualizador se utilizan distintos `BiquadFilterNode`.

3.7. Spotify Web API

Spotify Web API fue utilizada para incorporar una integración externa dentro de la aplicación.

La conexión se realiza utilizando OAuth y el flujo Authorization Code con PKCE.

La aplicación permite:

- Iniciar sesión con Spotify.
- Obtener información básica del usuario.
- Buscar canciones.
- Mostrar los resultados dentro de DeskForge.
- Mostrar portada, título, artista y álbum.
- Abrir una canción directamente en Spotify o en el navegador externo.

La autenticación utiliza un servidor local temporal en:

`http://127.0.0.1:8888/callback`

3.8. pnpm

pnpm fue utilizado como gestor de paquetes del proyecto.

La versión utilizada fue:

pnpm 11.20.0

Entre los comandos utilizados se encuentran:

`pnpm install`

`pnpm dev`

`pnpm build`

Las dependencias y scripts se administran mediante:

`package.json`

3.9. electron-builder

electron-builder fue utilizado para generar la versión instalable de DeskForge.

La versión utilizada fue:

electron-builder 26.15.3

La configuración se encuentra principalmente en:

electron-builder.yml

El formato elegido para Windows fue:

NSIS

Finalmente se generó un instalador con extensión:

.exe

4. Arquitectura de la solución

DeskForge utiliza la arquitectura característica de Electron, separando la aplicación en diferentes partes.

La estructura principal puede representarse de la siguiente manera:

DeskForge-BeatSync/

```
|
|
|— electron/
|  |— main.js
|  |— preload.js
|  └─ assets/
|
|— src/
|  |— index.html
|  |— main.js
|  |— services/
|  |  |— player.js
|  |  |— spotify.js
|  |  └─ equalizer.js
|  |
|  └─ styles/
|     └─ main.css
|
|— dist/
|— package.json
|— vite.config.mjs
```

|— electron-builder.yml

|— pnpm-lock.yaml

El archivo:

electron/main.js

corresponde al proceso principal de Electron.

Se encarga principalmente de:

- Crear la ventana.
- Crear el menú.
- Administrar el tray.
- Utilizar APIs nativas.
- Trabajar con archivos.
- Registrar atajos.
- Recibir el callback de Spotify.

El archivo:

electron/preload.js

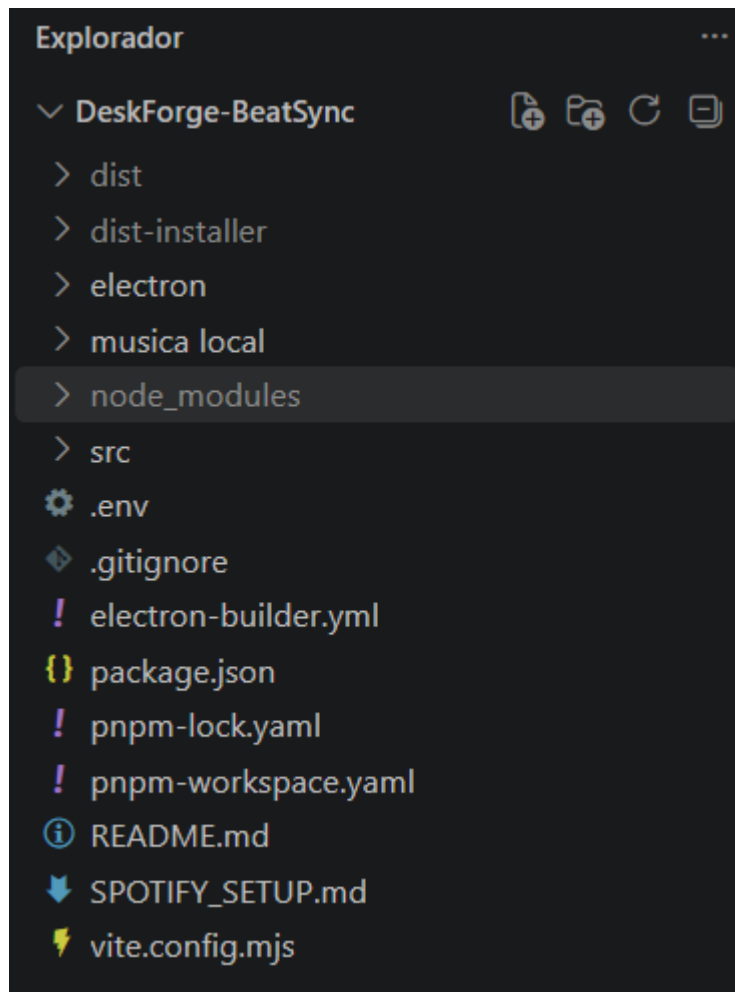
funciona como puente entre Electron y la interfaz.

Finalmente:

src/main.js

contiene gran parte de la lógica utilizada por el renderer para controlar las diferentes secciones de la aplicación.

Esta separación permite que la interfaz no tenga acceso directo a Node.js y que solamente pueda utilizar las funciones que se exponen mediante el preload.



5. Ventana principal y funcionalidades de escritorio

La aplicación posee una ventana principal redimensionable creada mediante `BrowserWindow`.

La interfaz contiene una barra lateral desde donde se puede acceder a las principales herramientas:

- Bloc de Notas.
- Pomodoro.
- Reproductor.

También contiene accesos a funciones como:

- Acerca de DeskForge.
- Minimizar a la bandeja del sistema.
- Cambio de tema.

Además de la interfaz personalizada, se creó un menú nativo en la parte superior de la aplicación.

El menú incluye diferentes opciones relacionadas con la aplicación y la ventana.

También se agregó un icono en la bandeja del sistema.

Desde este icono se puede:

- Mostrar la ventana.
- Ocultar la ventana.
- Acceder a información de la aplicación.
- Cerrar completamente DeskForge.

6. Bloc de Notas

Una de las herramientas de productividad incorporadas es el Bloc de Notas.

Este editor permite escribir contenido directamente dentro de la aplicación.

También utiliza diálogos nativos de Electron para trabajar con archivos.

Entre sus funciones se encuentran:

- Crear notas.
- Abrir archivos existentes.
- Guardar archivos.
- Guardar archivos con otro nombre.
- Mantener contenido automáticamente.

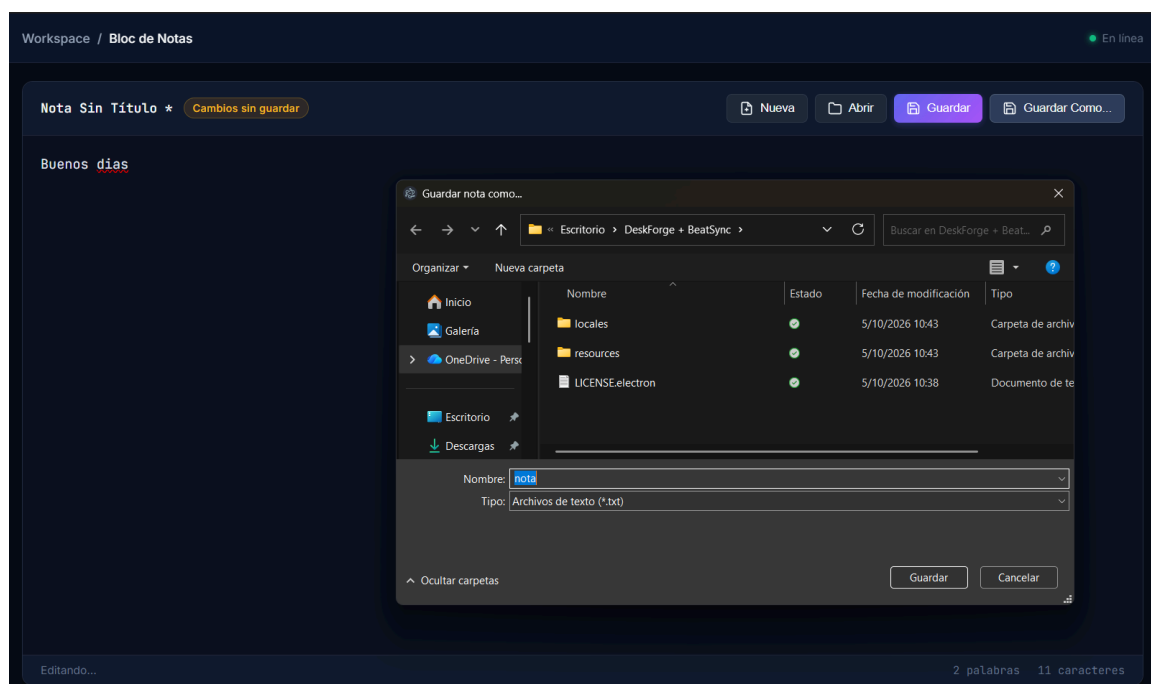
La comunicación con el sistema de archivos no se realiza directamente desde la interfaz.

En cambio, la interfaz utiliza IPC para solicitar al proceso principal que abra o guarde los archivos.

Esto permite mantener una separación más segura entre la interfaz y Node.js.

Los archivos pueden ser guardados principalmente como texto o Markdown.

Cuando una nota es guardada correctamente, la aplicación también puede mostrar una notificación nativa.



7. Temporizador Pomodoro

DeskForge incorpora un temporizador Pomodoro como segunda herramienta de productividad.

El sistema utiliza ciclos de trabajo y descanso.

El ciclo principal utiliza:

- 25 minutos de trabajo.
- 5 minutos de descanso.

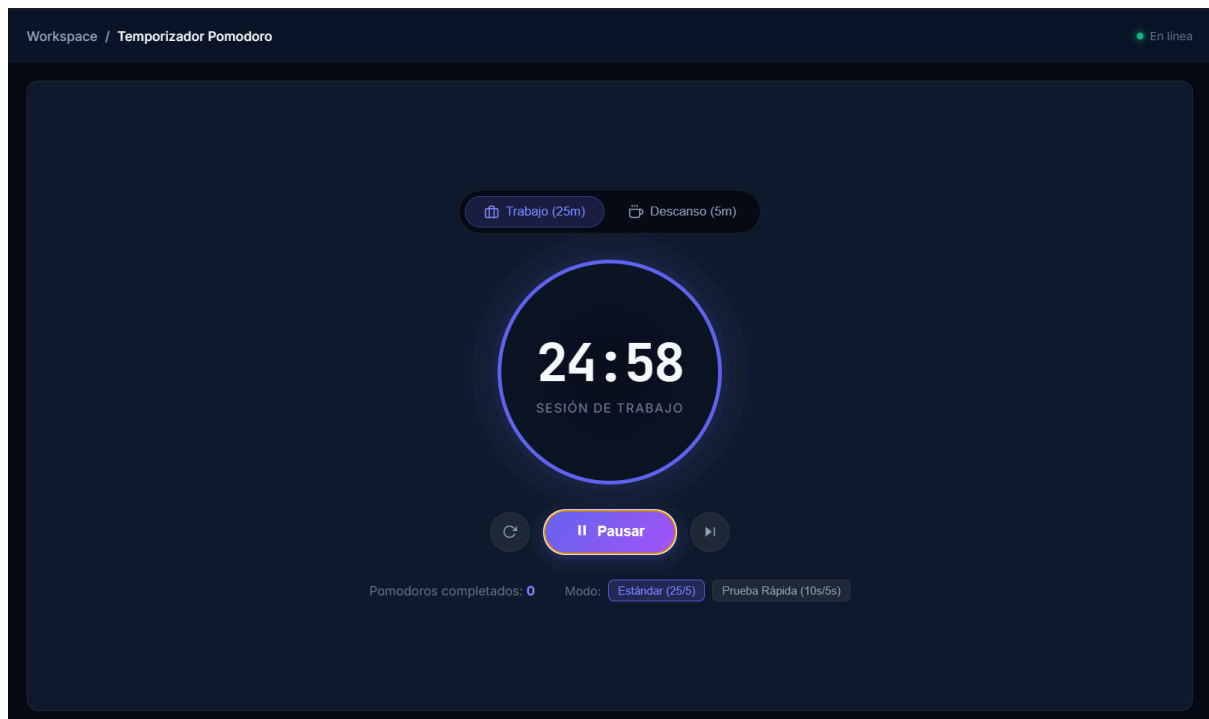
El usuario puede:

- Iniciar el temporizador.
- Pausarlo.
- Reiniciarlo.

Cuando el temporizador termina, se utiliza la API de notificaciones de Electron para mostrar un aviso nativo.

De esta forma, el usuario puede recibir la notificación incluso mientras trabaja con otras ventanas.

Además, el temporizador puede ser controlado mediante un atajo global incorporado posteriormente en el proyecto.



8. Reproductor musical

BeatSync funciona como el reproductor musical de DeskForge.

La reproducción se realiza mediante Howler.js.

La interfaz del reproductor incorpora información de la canción actual y diferentes controles.

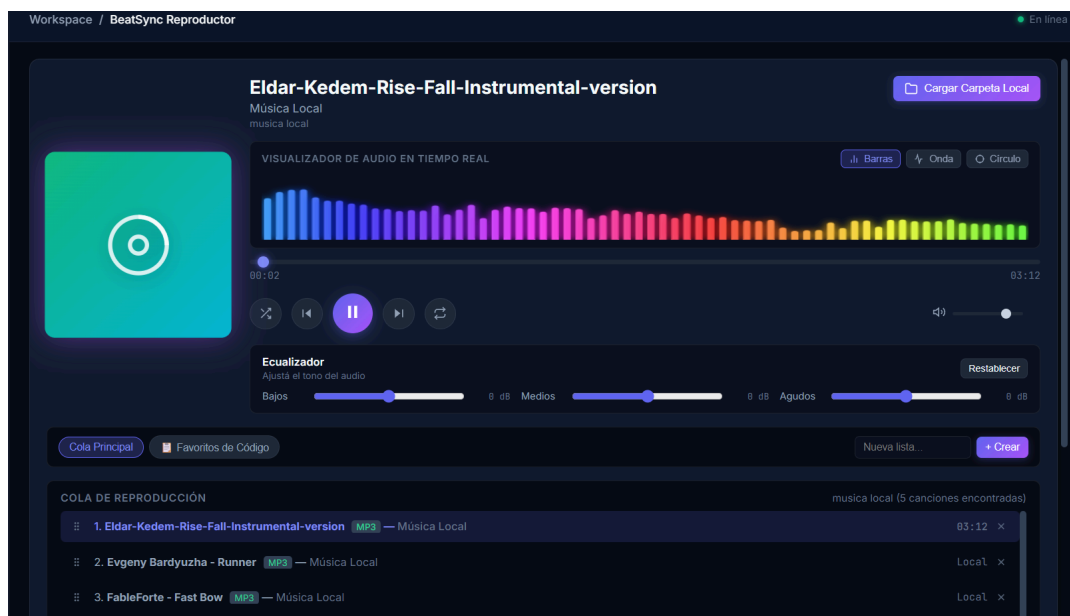
Entre ellos se encuentran:

- Reproducir.
- Pausar.
- Canción anterior.
- Canción siguiente.
- Control de volumen.
- Barra de progreso.
- Repetición.
- Reproducción aleatoria.

La aplicación también permite navegar entre las diferentes canciones cargadas.

El reproductor se encuentra integrado con otras funcionalidades del proyecto, principalmente:

- Visualizadores.
- Playlists.
- Ecualizador.



9. Carga de música local

Una de las funcionalidades principales de BeatSync es la posibilidad de seleccionar una carpeta del equipo y cargar las canciones almacenadas en ella.

La selección de carpeta se realiza mediante un diálogo nativo de Electron.

Después de seleccionar una carpeta, el proceso principal analiza los archivos existentes y filtra los formatos compatibles.

Los formatos permitidos son:

- .mp3
- .ogg
- .wav

Por cada archivo se obtiene información básica como:

- Nombre.
- Formato.
- Ruta.
- Carpeta de origen.

Después, las canciones son enviadas nuevamente a la interfaz.

Para permitir el acceso seguro a archivos locales desde Electron se implementó un protocolo personalizado llamado:

media://

Este protocolo permite que Howler pueda acceder a los archivos seleccionados sin exponer directamente rutas mediante mecanismos inseguros.

10. Visualizaciones de audio

BeatSync incorpora visualizaciones que reaccionan al audio en tiempo real.

Para desarrollarlas se utilizó Web Audio API.

El audio es analizado mediante:

AnalyserNode

Este nodo permite obtener información del sonido mientras la canción se está reproduciendo.

Se implementaron al menos dos modos diferentes de visualización.

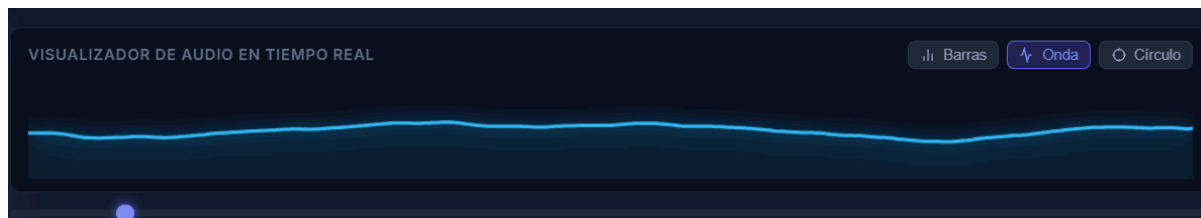
Uno de ellos representa las frecuencias del audio mediante barras.

Cada barra cambia su altura según la intensidad de una parte del espectro de frecuencias.

El segundo modo utiliza información de la forma de onda del audio.

Ambas visualizaciones se dibujan utilizando Canvas y se actualizan constantemente mientras se reproduce una canción.

Esto permite que la interfaz responda visualmente al audio.



11. Playlists

Otra funcionalidad implementada es el sistema de playlists.

El usuario puede organizar canciones dentro de listas personalizadas.

Las playlists permiten:

- Crear listas.
- Agregar canciones.
- Eliminar canciones.
- Reordenarlas.

Para el reordenamiento se utilizó Drag & Drop.

De esta manera, el usuario puede arrastrar una canción y cambiar su posición visualmente.

La información de las playlists se mantiene mediante almacenamiento local.

Esto permite conservarlas incluso después de cerrar la aplicación.

12. Integración con Spotify

DeskForge también incorpora integración con Spotify Web API.

Para utilizarla, el usuario debe iniciar sesión mediante su cuenta de Spotify.

El sistema utiliza OAuth con Authorization Code y PKCE.

Durante el proceso de autenticación, se abre Spotify para autorizar la aplicación.

Después de aceptar, Spotify redirige a:

`http://127.0.0.1:8888/callback`

Electron inicia un pequeño servidor local que recibe el código de autorización.

Este código es enviado posteriormente a la interfaz mediante IPC y se utiliza para solicitar un Access Token.

Una vez autenticado, DeskForge puede obtener información básica del usuario.

En la interfaz se muestra, por ejemplo:

Conectado como Alan

La aplicación también permite realizar búsquedas de canciones.

Cada resultado muestra información como:

- Portada.
- Nombre de la canción.
- Artista.
- Álbum.

También se encuentra disponible el botón:

Abrir en Spotify

Este botón utiliza Electron para abrir el enlace de la canción en Spotify o en el navegador predeterminado, evitando abandonar la interfaz de DeskForge.

La reproducción completa de Spotify no se incorporó dentro de la aplicación, ya que para cumplir con la integración requerida se implementó la búsqueda mediante su API.

INTEGRACIÓN EXTERNA

Spotify

Conectado como Alan

[Conectar Spotify](#) [Desconectar](#)

Ado [Buscar](#)

10 resultado(s) encontrado(s).

 RuLe Ado RuLe Abrir en Spotify	 AiAiA Ado AiAiA Abrir en Spotify	 Aishite Aishite Aishite Ado Ado's Utatemita Album Abrir en Spotify	 Usseewa Ado Kyougen Abrir en Spotify
 うっせえわ Ado うっせえわ Abrir en Spotify	 Crime And Punishment Ado Ado's Utatemita Album Abrir en Spotify	 Adore You Harry Styles Fine Line Abrir en Spotify	 Virgen Adolescent's Orquesta Ahora Más Que Nunca Abrir en Spotify
 I Adore You Ado Abrir en Spotify	 I Adore You Ado Abrir en Spotify		

13. Atajos globales

Se incorporaron atajos globales mediante Electron.

A diferencia de un atajo normal de una página web, un atajo global puede funcionar aunque la ventana de DeskForge no tenga el foco.

Se implementaron los siguientes:

Ctrl + Shift + Space

Permite reproducir o pausar la música.

Ctrl + Shift + P

Permite iniciar o pausar el temporizador Pomodoro.

Los atajos son registrados desde el proceso principal mediante:

`globalShortcut`

Cuando se activa uno, Electron envía un evento hacia el renderer utilizando IPC.

Al cerrar completamente la aplicación, los atajos son eliminados mediante:

`globalShortcut.unregisterAll()`

Esto evita que permanezcan registrados en Windows después de cerrar DeskForge.

14. Ecualizador

BeatSync incorpora un ecualizador de tres bandas.

Las bandas disponibles son:

- Bajos.
- Medios.
- Agudos.

Cada una puede ser modificada mediante un slider.

Los controles permiten aumentar o reducir aproximadamente:

-12 dB a +12 dB

El ecualizador fue desarrollado utilizando Web Audio API.

Para modificar el sonido se utilizaron BiquadFilterNode.

Los filtros principales utilizados son:

- lowshelf para bajos.
- peaking para medios.
- highshelf para agudos.

Los filtros forman parte de la cadena de procesamiento de audio antes de que la señal llegue a la salida.

También se agregó un botón que permite restablecer todos los valores a 0 dB.

Las configuraciones del ecualizador se almacenan localmente para mantener los valores seleccionados.

15. Tema claro y oscuro

La aplicación incorpora un sistema de tema claro y oscuro.

Durante la primera ejecución se puede detectar el tema configurado en el sistema operativo mediante:

`prefers-color-scheme`

Si el usuario selecciona manualmente otro tema, la preferencia se almacena utilizando:

`localStorage`

De esta forma, la próxima vez que se abra DeskForge se recupera la selección anterior.

El cambio visual se implementa principalmente mediante variables CSS.

Esto permite cambiar elementos como:

- Fondos.
- Tarjetas.
- Textos.
- Bordes.
- Inputs.
- Botones.

sin tener que duplicar toda la hoja de estilos.

16. Empaquetado para Windows

Una vez terminadas las funcionalidades de la aplicación se realizó el empaquetado para Windows.

Para esto se utilizó:

electron-builder

La configuración del empaquetado se encuentra en:

electron-builder.yml

El instalador utiliza NSIS.

La compilación se realizó mediante:

pnpm build

Primero Vite genera la versión de producción dentro de:

dist/

Posteriormente electron-builder empaqueta la aplicación y genera los archivos correspondientes dentro de:

dist-installer/

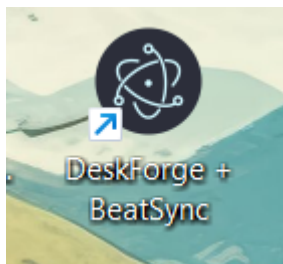
Entre los archivos generados se encuentra:

DeskForge + BeatSync-Setup-1.0.0.exe

Este archivo permite instalar DeskForge como una aplicación normal de Windows.

Durante la instalación se pueden crear accesos directos y seleccionar la ubicación correspondiente.

Después de instalar la aplicación se verificó que pudiera ejecutarse independientemente del entorno de desarrollo.



17. Seguridad e IPC

Uno de los puntos importantes del proyecto fue mantener separadas las responsabilidades entre el proceso principal y la interfaz.

La ventana de Electron utiliza:

`contextIsolation: true`

y:

`nodeIntegration: false`

Esto evita que la interfaz tenga acceso directo a Node.js.

Las funciones necesarias se exponen mediante:

`electron/preload.js`

utilizando:

`contextBridge`

Por ejemplo, la interfaz puede solicitar:

- Abrir archivos.
- Guardar archivos.
- Seleccionar una carpeta de música.
- Mostrar notificaciones.
- Abrir enlaces externos.
- Recibir eventos de Spotify.
- Recibir atajos globales.

La comunicación entre procesos se realiza principalmente con:

`ipcRenderer.invoke()`

`ipcMain.handle()`

y eventos enviados desde el proceso principal hacia el renderer.

Esta estructura permite mantener una separación más segura y ordenada.

18. Pruebas de funcionamiento

Durante el desarrollo se realizaron diferentes pruebas para comprobar el funcionamiento de DeskForge.

En primer lugar se inició el entorno de desarrollo mediante:

```
pnpm dev
```

Se comprobó que Vite iniciara correctamente y que Electron pudiera abrir la ventana principal.

Después se probaron individualmente las principales funcionalidades.

En el Bloc de Notas se verificó:

- Escritura de texto.
- Apertura de archivos.
- Guardado.
- Guardar como.
- Notificaciones.

En el Pomodoro se verificó:

- Inicio.
- Pausa.
- Reinicio.
- Notificación al finalizar.

En el reproductor se verificó:

- Carga de canciones.
- Play y pausa.
- Canción anterior y siguiente.
- Volumen.
- Progreso.
- Repeat.
- Shuffle.

También se seleccionaron carpetas locales con archivos MP3, OGG y WAV.

Se comprobó que las visualizaciones reaccionaran correctamente mientras se reproducía música.

También se probaron las playlists, verificando la creación y el reordenamiento mediante Drag & Drop.

En Spotify se verificó:

- Autenticación.
- Obtención del perfil.
- Búsqueda.
- Visualización de resultados.
- Apertura externa de canciones.

Para los atajos globales se comprobó que funcionaran aunque DeskForge no tuviera el foco.

También se modificaron las tres bandas del ecualizador para verificar el cambio de sonido.

Finalmente se probó el tema claro y oscuro, comprobando que la selección permaneciera almacenada después de cerrar y volver a abrir la aplicación.

La compilación final se realizó mediante:

```
pnpm build
```

Vite compiló correctamente la aplicación y electron-builder generó el instalador de Windows.

Luego se instaló DeskForge mediante el archivo .exe y se comprobó que la aplicación instalada funcionara correctamente.

19. Dificultades encontradas

Durante el desarrollo se presentaron diferentes problemas, principalmente relacionados con Spotify, Electron y el empaquetado.

Uno de los primeros problemas de la integración con Spotify fue la lectura del Client ID almacenado en el archivo:

```
.env
```

La configuración de Vite utilizaba:

```
root: 'src'
```

por lo que inicialmente el archivo .env ubicado en la raíz del proyecto no era detectado.

El problema se solucionó agregando en:

```
vite.config.mjs
```

la configuración:

```
envDir: '..'
```

De esta manera Vite pudo leer correctamente las variables de entorno desde la raíz del proyecto.

Otro problema apareció durante el flujo OAuth de Spotify.

Después de autorizar la cuenta, el servidor local utilizado para recibir el callback se cerraba demasiado rápido.

Esto provocaba errores como:

```
ERR_CONNECTION_REFUSED
```

El problema se solucionó manteniendo el servidor activo durante la ejecución de la aplicación en lugar de cerrarlo inmediatamente después de recibir el primer callback.

También aparecieron errores por bloques duplicados dentro del manejo del callback de Spotify.

En una de las pruebas existían dos declaraciones de:

```
handleSpotifyAuthCode
```

y:

`handleSpotifyAuthError`

Esto provocaba errores de JavaScript relacionados con la redeclaración de variables.

Se eliminó el bloque duplicado y se mantuvo una única implementación.

Posteriormente apareció un error de sintaxis dentro de:

`electron/main.js`

El problema se debía a que faltaba cerrar correctamente el callback de:

`http.createServer()`

Esto provocaba:

`SyntaxError: missing ) after argument list`

Después de agregar el cierre correspondiente, Electron volvió a iniciar correctamente.

Otro problema importante apareció cuando Spotify intentaba solicitar el Access Token.

La interfaz mostraba:

`Failed to fetch`

El problema no estaba en las credenciales, sino en la política Content Security Policy del documento.

La directiva `connect-src` no permitía conexiones hacia:

`accounts.spotify.com`

y:

`api.spotify.com`

Después de agregar ambos dominios a la política, la autenticación y las búsquedas comenzaron a funcionar correctamente.

También fue necesario corregir la forma en que se abrían los resultados de Spotify.

Inicialmente, el enlace se abría dentro de Electron y reemplazaba la interfaz de DeskForge.

La solución fue utilizar:

```
shell.openExternal()
```

mediante IPC.

De esta forma las canciones se abren externamente y DeskForge permanece abierto.

Durante el empaquetado apareció otro problema.

Al ejecutar:

```
pnpm build
```

Vite terminaba correctamente, pero electron-builder fallaba durante la generación del instalador NSIS con:

```
spawn UNKNOWN
```

El proyecto se encontraba originalmente dentro de una carpeta sincronizada por OneDrive.

Para solucionar el problema se realizó una copia del proyecto fuera de OneDrive, utilizando una ruta similar a:

```
C:\Proyectos\DeskForge-BeatSync
```

Desde esa ubicación electron-builder pudo terminar correctamente el proceso y generar el instalador .exe.

Estos problemas permitieron comprender mejor la relación entre Electron, el renderer, las políticas de seguridad, OAuth y las herramientas utilizadas durante el empaquetado.

20. Resultado final

Como resultado final se obtuvo una aplicación de escritorio funcional denominada DeskForge + BeatSync.

Actualmente la aplicación permite:

- Ejecutarse como aplicación de escritorio en Windows.
- Utilizar una ventana redimensionable.
- Utilizar menú nativo.
- Utilizar bandeja del sistema.
- Crear y editar notas.
- Abrir y guardar archivos.
- Utilizar un temporizador Pomodoro.
- Mostrar notificaciones nativas.
- Cargar música desde una carpeta local.
- Reproducir MP3, OGG y WAV.
- Controlar play y pausa.
- Cambiar de canción.
- Controlar volumen.
- Utilizar reproducción aleatoria.
- Utilizar repetición.
- Mostrar el progreso de reproducción.
- Visualizar el audio en tiempo real.
- Cambiar entre diferentes visualizaciones.
- Crear playlists.
- Reordenar canciones mediante Drag & Drop.
- Mantener playlists mediante almacenamiento local.
- Iniciar sesión con Spotify.
- Buscar canciones mediante Spotify Web API.
- Mostrar portada, artista y álbum.
- Abrir canciones externamente en Spotify.
- Utilizar atajos globales.
- Modificar bajos, medios y agudos mediante un ecualizador.
- Cambiar entre tema claro y oscuro.
- Mantener la preferencia del tema.
- Generar una versión de producción.
- Generar un instalador para Windows.

La versión final fue compilada utilizando:

```
pnpm build
```

El resultado del empaquetado fue:

DeskForge + BeatSync-Setup-1.0.0.exe

Este instalador permitió instalar DeskForge y utilizarlo fuera del entorno de desarrollo.

21. Conclusión

El desarrollo de DeskForge + BeatSync permitió trabajar con diferentes conceptos relacionados con aplicaciones de escritorio y tecnologías web dentro de un mismo proyecto.

Uno de los principales conceptos trabajados fue la arquitectura de Electron y la separación entre main, preload y renderer.

El uso de IPC permitió comunicar la interfaz con funciones propias del sistema operativo sin habilitar acceso directo a Node.js desde el renderer.

También se trabajó con APIs nativas de Electron mediante ventanas, diálogos, notificaciones, bandeja del sistema, enlaces externos y atajos globales.

En la parte musical, Howler.js permitió implementar el reproductor y manejar diferentes formatos de audio.

Por otra parte, Web Audio API permitió analizar el sonido en tiempo real para desarrollar visualizaciones y un ecualizador de tres bandas.

La integración con Spotify permitió trabajar con una API externa real y utilizar OAuth mediante Authorization Code con PKCE.

Además, las playlists, el almacenamiento local y el sistema de temas permitieron mejorar la experiencia de uso y conservar diferentes configuraciones del usuario.

Finalmente, electron-builder permitió convertir el proyecto en una aplicación instalable para Windows mediante un archivo .exe.

En conjunto, el proyecto permitió combinar desarrollo web, aplicaciones de escritorio, APIs externas, audio y funciones del sistema operativo, obteniendo como resultado una aplicación funcional que reúne herramientas de productividad y reproducción musical dentro de una misma interfaz.